Assignment 1: Inter-AS Routing with ns-3 (BGP-style)

Course: Advanced Computer Networks (COL724/7524)

Overview

This assignment introduces the idea of inter-AS (Autonomous System) routing using a simplified BGP simulation in ns-3.

You are provided with a baseline script (`bgp.cc`) that sets up four routers (AS1–AS2–AS3–AS4) connected in a line, with a different bandwidth/delay on each hop, and sends UDP traffic end-to-end across them.

Your tasks are to extend this baseline into more realistic inter-domain situations by adding links, policies, and failures. Each part after A must be implemented as a **separate script** (e.g. `bgp_partB.cc`, `bgp_partC.cc`, `bgp_partD.cc`).

Optional (not required, but can strengthen your report): ns-3's NetAnim module (the `AnimationInterface` class) can visualize packet flow across your topology as an animation. None of the provided/baseline code uses it, and it is not required for any part. If you choose to add it to your own scripts, screenshots or a short recording of the animation showing traffic taking the expected path (e.g. the reroute in Parts D/E) make for a more compelling report, and you may be asked to walk through them at the viva.

Part 0 (Background Research — Real BGP Data)

Before running any simulations, investigate the real BGP connectivity of your own institute using online BGP monitoring tools. (If your institute's prefix is not visible on public looking glasses, pick any NKN- or ERNET-connected Indian academic/research network instead, and say which one you picked and why.)

(a) Use at least **two different** public BGP looking glasses / route servers (e.g. bgp.he.net, RIPEstat, NLNOG Looking Glass, bgpview.io) — not just one.

(b) Search for the institute's IP prefix or ASN on both tools.

(c) Record: which **upstream ISPs** announce the institute's routes to the global Internet, and from which providers those ISPs themselves receive announcements (follow the chain at least four steps further).

(d) Compare the two looking-glass tools: do they report the same upstream path? Note and briefly explain any discrepancy you find.

(e) Summarize the findings in a short paragraph and include screenshots of the BGP route views from both tools.

Deliverable: A 1–2 page write-up explaining the institute's real-world BGP peers, the upstream connectivity hierarchy you observed, and the cross-tool comparison.

Part A (Baseline — Provided)

- Run the given `bgp.cc`.
- Confirm packets flow along the path AS1 → AS2 → AS3 → AS4.
- Observe and explain the console log.
- Using the per-link delays hard-coded in the script, compute the theoretical minimum one-way propagation delay across the full path, and compare it with the average delay reported by FlowMonitor. Briefly explain any gap between the two.
- Submit the log output, the generated file `interas-baseline-results.xml`, and your delay calculation.

Note: This part is provided as a warm-up. You do not need to modify the code — `bgp.cc` already installs FlowMonitor and writes `interas-baseline-results.xml` on its own. To get the average delay, open that XML and read the single `<Flow>` block's `delaySum` and `rxPackets` attributes (average delay = `delaySum` ÷ `rxPackets`); a short separate script to parse the XML, or just reading it by hand, is fine and does not count as modifying `bgp.cc`.

Part B (New Script Required)

Add a direct peering link and compare paths

- Create a new script (`bgp_partB.cc`).
- Add a **direct link between AS1 and AS4**, parallel to the AS1–AS2–AS3–AS4 chain.
- Configure the new link with **higher bandwidth and lower delay** than any individual link in the chain.
- Recompute routes and re-run the simulation.
- Show which path the traffic now takes, and explain why in terms of both hop count and the metric ns-3's global routing actually uses.
- Report the **percentage reduction** in average end-to-end delay compared to your Part A measurement.

Deliverables: script + console log excerpt + XML results + delay comparison.

Part C (New Script Required)

Simulate a policy preference (Local-Pref analogy)

- Create a new script (`bgp_partC.cc`), starting from Part B's topology.
- Even though the AS1–AS4 direct link is faster, implement a policy so that AS1 prefers sending traffic via the AS2–AS3 chain.
- Demonstrate this using **two different mechanisms**, and briefly compare them:
  - A **static route** on AS1 that forces traffic via AS2, and
  - An adjustment of the **interface metric/cost** so that the AS1–AS2–AS3–AS4 path is chosen by global routing.
- Re-run and demonstrate, for each mechanism, that packets follow AS1 $\rightarrow$ AS2 $\rightarrow$ AS3 $\rightarrow$ AS4.

Deliverables: script(s) + a short explanation of how each mechanism enforces the policy + logs for both.

Part D (New Script Required)

Failure and recovery study

- Create a new script (`bgp_partD.cc`), starting from Part B's topology (chain + direct link).
- Introduce a **link failure** on either AS1–AS4 or AS3–AS4 (choose one, and justify your choice based on which link is actually carrying traffic just before the failure).
- The failure should start at $t = 6\,\text{s}$ and last for 3 seconds.
- Ensure that if an alternate path exists, traffic continues; if not, show the outage.
- Use FlowMonitor to measure, separately for the three intervals (before / during / after the failure):
  - Packets delivered vs. lost
  - Average end-to-end delay
  - Jitter (delay variation)
- Plot **delay vs. time** across the three intervals (a hand-drawn sketch, spreadsheet chart, or script-generated plot is fine).

Deliverables: script + plot + short timeline/analysis of results.

Part E (New Script Required)

Dual-path rerouting study

- Create a new script (`bgp_partE.cc`). This uses a fresh 5-router topology, independent of Parts A–D.
- Design the topology so there are **two distinct paths** between R1 and R5, of different lengths:
  - Path A: R1–R2–R3–R5 (3 hops)
  - Path B: R1–R4–R5 (2 hops)
- Before introducing any failure, identify (and justify) which path carries traffic by default.
- At time $t = 6\,\text{s}$, fail the link that is currently carrying traffic.
- At time $t = 9\,\text{s}$, restore the failed link.
- Send **continuous traffic** from R1 to R5 throughout the experiment.
- Use FlowMonitor to measure:
  - Packet delivery and loss during failure and recovery
  - Route changes observed after link failure and restoration
  - **Convergence time:** the time between the failure (or restoration) event and the first successfully delivered packet on the new (or restored) path

Deliverables: new script + results (graphs or tables of delivery/loss, route changes, and convergence time).

Submission Guidelines

- Submit a single compressed archive (`.zip`) containing:
  - All source code files for Parts B, C, D, and E.
  - Output logs, XML results, and graphs/tables as required by each part.
  - Your write-up for Part 0.
- The zip file must be named in the following format: `COL<course_code>_assignment1_1_<entry_n` for the first submission and `COL<course_code>_assignment1_2_<entry_number>.zip` for the second submission (for example: `COL724_assignment1_1_2026MT1234.zip`).
- Ensure that each script is clearly named (`bgp_partB.cc`, `bgp_partC.cc`, etc.) and runs without modification in the provided ns-3 environment.

Administrative Details

Deadline: 17/08/2026 for Parts 0, A, B, C
Deadline: 27/08/2026 for Parts D, E
Work policy: This assignment must be completed *individually*. Collaboration on ideas is permitted at a high level, but sharing code or write-ups is strictly prohibited.